



Soru 1

Puan: 5,00

Aşağıdakilerden hangisi FCFS algoritmasının problemidir?

- A ☐ Okuma yazma hatası
- B ☐ Meşgul bekleme
- C ☐ Bellek yazma hatası
- D ☒ Konvoy etkisi
- E ☐ Senkronizasyon

Soru 2

Puan: 5,00

Aşağıdakilerden hangisi proses senkronizasyon yöntemlerinden değildir?

- A ☒ Monitör
- B ☐ Sınırlı tapon
- C ☐ Muteks kilidi
- D ☐ Semafor
- E ☐ Peterson çözümü

Soru 3

Puan: 5,00

Aşağıdakilerden hangisi proses senkronizasyonu ile çözülebilir?

- A ☐ Yarış durumundaki proseslerin yönetilmesi
- B ☐ Paylaşılan veriye erişim
- C ☐ Eş zamanlı erişimden kaynaklı problemler
- D ☐ Paralel erişimden kaynaklı problemler
- E ☒ hepsi

Soru 4

Puan: 5,00

Aşağıdakilerden hangisi kritik bölge probleminin çözüm şartlarından?

- I. Karşılıklı dışlama (mutual exclusion) II. İlerleme (progress)
- III. Sınırlı bekleme (bounded waiting) IV. Çevrimsel bekleme (circular waiting)

- A ☐ I ve II
- B ☒ I, II ve III
- C ☐ I, II, III ve IV
- D ☐ I, II ve III
- E ☐ sadece I
- F ☐ sadece IV

Soru 5

Puan: 5,00

- i. do {
- ii. flag[0] = 1;
- iii. turn = 1;
- iv. while (flag[i] == 1 && turn == 1);
- v. //kritik bölge
- vi.;
- vii. //kalan bölge
- viii. } while (TRUE);

Sınırlı bekleme (bounded waiting) gereksinimini karşılamak için vi. satıra hangi kod parçası gelmelidir?

- A ☐ flag[i]=1;
- B ☐ turn=0;
- C ☐ turn=1;
- D ☒ flag[0]=0;
- E ☐ flag[i]=1;

Soru 6

Puan: 5,00

Aşağıdakilerden hangisi semafor'u en iyi tanımlar?

- A ☐ Yalnız yazılım tarafından desteklenen özel bir değişkendir
- B ☐ Yalnız donanım tarafından desteklenen özel bir değişkendir
- C ☐ Tamsayı değeri alamayan özel bir değişkendir
- D ☐ Yazılımının tipini tanımlamadığı özel bir değişkendir
- E ☒ Proses senkronizasyonu için işletim sistemi içinde gerçekleştirilmiş özel bir değişkendir

Soru 7

Puan: 5,00

X, Y ve Z prosesleri a, b, c ve d semaforlarında döngü içinde paylaşılan bir değişkene erişmek istiyor. Tüm semaforlar binary olup, hepsinin başlangıç değeri 1 dir. Aşağıdakilerden hangisinde deadlock oluşmaz?

- A ☐ X wait(a) wait(b) wait(c)
Y wait(b) wait(c) wait(d)
Z wait(c) wait(d) wait(a)
- B ☐ X wait(a) wait(b) wait(c)
Y wait(b) wait(c) wait(d)
Z wait(c) wait(d) wait(a)
- C ☐ X wait(b) wait(c) wait(a)
Y wait(a) wait(b) wait(c)
Z wait(c) wait(d) wait(d)
- D ☒ X wait(a) wait(c) wait(c)
Y wait(b) wait(b) wait(d)
Z wait(c) wait(d) wait(a)
- E ☐ X wait(b) wait(b) wait(a)
Y wait(a) wait(c) wait(c)
Z wait(c) wait(d) wait(a)

Soru 8

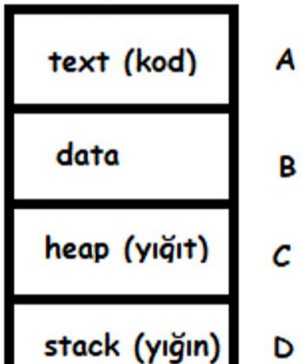
Puan: 5,00

İş parçacıkları (threads) ve prosesler ile ilgili olarak aşağıdakilerden hangisi yanlıştır?

- A ☐ İş parçacıkları arasında iletişim paylaşılan değişkenler aracılığı ile yapılabilir.
- B ☒ İş parçacıklarının aynı adres alanı içinde özel (paylaşımsız) yığınları (stack) vardır.
- C ☐ Hem İş parçacıkları ve hem de prosesler bağımsız iş süreçleri olarak organize edilebilirler.
- D ☐ Her iş parçacığının proses adres alanı dışında kendine ait bir kod bölgesi (sayfası) vardır.
- E ☐ İş parçacıkları aynı bellek alanını kullanır, prosesler ise ayrı adres alanlarını kullanırlar.

Soru 9

Puan: 5,00



Yukardaki şekilde gösterilen bir prosenin bellekteki bölgelerine göre aşağıdakilerden hangisi doğrudur?

- A
- ```
#include<stdio.h>

int a; A

int main() {
 int b; B
 int *c = malloc(16); C
 b = b + 25;
 return 0;
}
```
- B
- B
- ```
#include<stdio.h>

int a; B

int main() {
    int b; A
    int *c = malloc(16); C
    b = b + 25;
    return 0;
}
```
- D
- C
- ```
#include<stdio.h>

int a; B

int main() {
 int b; D
 int *c = malloc(16); C
 b = b + 25;
 return 0;
}
```
- A
- D
- ```
#include<stdio.h>

int a; B

int main() {
    int b; C
    int *c = malloc(16); D
    b = b + 25;
    return 0;
}
```
- A
- E
- ```
#include<stdio.h>

int a; A

int main() {
 int b; B
 int *c = malloc(16); C
 b = b + 25;
 return 0;
}
```
- D

#### Soru 10

Puan: 5,00

Aşağıdakilerden hangisi bir prosesler arası senkronizasyon için kullanılacak bir yöntem değildir?

- A ☐ Kesmelerin serbest bırakılması
- B ☐ Semafor
- C ☒ Atomic komutlar (Test&Set)
- D ☐ Mutex & Lock
- E ☐ Monitör

#### Soru 11

Puan: 5,00

- i. do {
- ii. flag[0] = 1;
- iii. turn = 1;
- iv. while (flag[i] == 1 && turn == i);
- v. //kritik bölge
- vi. ....}

vii. `//kalan bölge`  
viii. `} while (TRUE);`

Karşılıklı dışlama hangi satırda gerçekleştirilmiştir?

- A ☐ ii  
B ☐ iv  
C ☐ i  
D ☐ iii  
E ☒ vi

#### Soru 12

Proses anahtarlama (context switch) çok sık olursa, kayıp zaman artar.

Puan: 5,00

- A ☒ Doğru  
B ☐ Yanlış

#### Soru 13

Çoklu kaynak durumunda aşağıdaki algoritmalarından hangisi deadlock (ölümcül kilit) için kullanılabilir?

Puan: 5,00

- A ☒ Banker algoritması  
B ☐ Peterson algoritması  
C ☐ Safe state algoritması  
D ☐ Dijkstra metodu  
E ☐ Wait-for graf yöntemi,

#### Soru 14

Öncelikli İş Sıralama/Planlama algoritmasında Açlık probleminin çözümü nedir?

Puan: 5,00

- A ☒ Yaşlandırma  
B ☐ Mesgul Bekleme  
C ☐ Bir sonraki CPU Patlama zamanı tahmini  
D ☐ Ölümcül Kilitlenme  
E ☐ Karşılıklı Dışlama

#### Soru 15

Aşağıdakilerden hangisi Proses Kontrol Bloğunda (PCB) yer almaz?

Puan: 5,00

- A ☒ Global değişkenler  
B ☐ Proses numarası  
C ☐ Çözeltme (planlama) bilgisi  
D ☐ Program sayacı  
E ☐ Proses durum bilgisi

#### Soru 16

Aşağıdakilerden hangisi ölümcül-kilit (deadlock) oluşma koşullarından biri değildir?

Puan: 5,00

- A ☐ Bir proses başka kaynaklar beklerken, kendisine daha önceden tahsis edilen kaynakları tutabilir (hold and wait).

- B** ☐ Bir anda, bir kaynağı sadece bir proses tutabilir (mutual exclusion).
- C** ☐ Proseslerden oluşan bir kapalı çevrim vardır öyle ki; her bir proses bir kaynak tutarken diğer bir prosesin kaynağını talep eder (circular wait).
- D** ☒ Bir kaynağa ihtiyaç olduğunda, onu tutan proses bırakılabilir (preemption).
- E** ☐ hepsi

#### Soru 17

Puan: 5,00

Aşağıda verilen CPU Patlama zamanlarına göre en uygun kuantum zamanı (q) aşağıdaki verilenlerden hangisidir?

2, 10, 3, 5, 5, 1, 3, 6, 4, 9

- A** ☒ 6
- B** ☐ 11
- C** ☐ 2
- D** ☐ 1
- E** ☐ 3

#### Soru 18

Puan: 5,00

Threadler (iş parçacığı) için aşağıdakilerden hangisi doğrudur?

- A** ☒ Kod ve data paylaşılr, yığıt ve yığın her bir thread'de ayrıdır.
- B** ☐ Kod, data ve yığın paylaşılr, yığıt her bir thread'de ayrıdır
- C** ☐ Kod ve yığın her bir thread için özeldir, data ile yığıt ise thread'ler arasında paylaşılr.
- D** ☐ Kod, data, yığıt (heap) ve yığın (stack) tüm thread'lerce paylaşılr.
- E** ☐ Kod, data ve yığıt paylaşılr, yığın her bir thread'de ayrıdır.

#### Soru 19

Puan: 5,00

Aşağıdakilerden hangisi proses veya thread senkronizasyonunda kullanılan yöntemlerden biri değildir?

- A** ☒ Rollback (geri sarma)
- B** ☐ Monitörler
- C** ☐ Mutex-locks (mutex-kilitler)
- D** ☐ test&set(), compare&swap() gibi atomik komutlar
- E** ☐ Semaforlar

#### Soru 20

Puan: 5,00

Tek parça bir yapıya sahip işletim sistemi türü aşağıdakilerden hangisidir?

- A** ☐ Hibrit Sistemler
- B** ☐ Mikro Çekirdek
- C** ☐ Katmanlı Yapı
- D** ☐ Modüler Yapı
- E** ☒ Monolitik

Sınava Geri Dön

Sınava Bitir